

Diseño de la Base de Datos MongoDB

Actividad N°: 12 **Fecha:** 16/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Diseñar el modelo de datos en MongoDB: colecciones, campos, relaciones (referencias) e índices, alineado con los requerimientos funcionales y de rendimiento definidos previamente.

2. Modelo de colecciones



```
timestamps (createdAt, updatedAt)
```

3. Justificación de diseño por colección

Ticket

- Los nombres de campo replican literalmente las columnas del CSV de origen (**First Name**, **Seat**, **Ticket ID**, etc.) para simplificar la importación directa sin necesidad de mapear/renombrar cada columna.
- **Ticket ID** es **unique** e indexado: es la clave natural de negocio para evitar boletos duplicados en reimportaciones.
- Los campos de canje (**canjeado**, **fechaCanje**, **usuarioCanje**, **puntoCanje**) están separados de los campos de impresión (**impreso**, **fechaImpresion**, **usuarioResponsable**) para permitir distinguir ambos eventos en el ciclo de vida del boleto.
- **reimpresiones** se modela como subdocumento embebido (arreglo), ya que su ciclo de vida depende completamente del ticket padre y no requiere consultarse de forma independiente.

User

- **puntoTrabajo** es condicionalmente requerido solo si **rol** = **'staff'**, reflejando que un Jefe no está atado a un único punto físico.
- **creadoPor** es una referencia a sí misma (self-reference), permitiendo trazar qué Jefe creó cada cuenta.
- La contraseña nunca se expone: se hashea en un hook **pre('save')** y se excluye explícitamente en **toJSON()**.

PuntoVenta

- **localidades** es un arreglo de strings **sin enum fijo**, por diseño: el valor proviene de datos reales del evento (columna Seat del CSV) y cambia de un concierto a otro.

AuditLog

- **tipo** sí usa **enum** fijo, porque los tipos de operación auditable son controlados por el propio sistema (no por datos externos variables como las localidades).
- **detalles** usa **Mixed** para admitir distinta forma de datos según el tipo de operación (ej. un canje individual vs. un canje masivo con contador de tickets).

4. Diseño de índices (rendimiento)

Colección	Índice	Propósito
Ticket	Ticket ID (unique)	Búsqueda directa y prevención de duplicados
Ticket	First Name + Last Name	Búsqueda por nombre completo
Ticket	Seat	Filtro por localidad
Ticket	Numero de Cedula	Búsqueda por cédula

Colección	Índice	Propósito
Ticket	<code>impreso + puntoTrabajo, canjeado + puntoTrabajo</code>	Filtros combinados frecuentes en la operación diaria
Ticket	<code>updatedAt (desc), Ticket + updatedAt</code>	Sincronización/verificación de cambios recientes
PuntoVenta	<code>nombre, activo</code>	Listado y validación de unicidad
AuditLog	<code>tipo + createdAt, usuario + createdAt, ticketId + tipo</code>	Consultas de auditoría por tipo, por usuario o por boleto

5. Relaciones entre colecciones

- `Ticket.usuarioResponsable, Ticket.usuarioCanje, Ticket.reimpresiones[].usuario` → referencian `User._id`.
- `PuntoVenta.creadoPor` → referencia `User._id`.
- `AuditLog.usuario` → referencia `User._id`.
- No existe relación directa `Ticket ↔ PuntoVenta` por `_id`; la asociación es lógica, a través del valor de `Seat/puntoTrabajo` (string), lo que evita depender de un ID fijo cuando cambian las localidades entre eventos.

6. Conclusiones del día

El modelo de datos queda diseñado en 4 colecciones con relaciones claras, priorizando búsquedas rápidas mediante índices y manteniendo flexible el campo de localidades para no atar el sistema a un evento específico.

Observaciones: Sin observaciones.